

Q_Learning_CIA_Test_1_149

October 5, 2026

1 Q-Learning CIA Test 1 — FrozenLake Debugging

Marks:15 ## Instructions

The following program implements a **Q-learning delivery robot using a custom 4×4 Frozen-Lake environment**.

- Start state = 3
- Goal state = 12
- Correct the errors without completely rewriting the program.
- Execute the corrected program and verify the learned route and final reward.

1.0.1 Environment

```
F F F S
F F H F
F F F F
G F F F
```

State numbering is row-wise from 0 to 15.

```
[1]: # Q-Learning using a custom FrozenLake environment

import numpy as np
import random
import gymnasium as gym

custom_map = [
    "FFFS",
    "FFHF",
    "FFFF",
    "GFFF"
]

env = gym.make(
    "FrozenLake-v1",
    desc=custom_map,
    is_slippery=False # FIX: deterministic moves so the learned route can be
    ↪verified
)
```

```

n_states = env.observation_space.n
n_actions = env.action_space.n

Q_table = np.zeros((n_states, n_actions)) # FIX: start Q-values at 0

learning_rate = 0.8 # FIX: 0.01 is too slow to learn
discount_factor = 0.95 # FIX: 0.5 discounts the distant goal too heavily
epsilon = 0.2
episodes = 1000 # FIX: 100 episodes is too few to learn the route

# 0 = Left, 1 = Down, 2 = Right, 3 = Up

def choose_action(state):
    if random.random() < epsilon: # FIX: explore with probability epsilon
        return random.randrange(n_actions)
    else:
        return np.argmax(Q_table[state]) # FIX: exploit the best (max) action

for episode in range(episodes):

    state, info = env.reset()

    for step in range(100):

        action = choose_action(state)

        next_state, reward, terminated, truncated, info = env.step(action)

        done = terminated or truncated # FIX: episode ends on either

        best_next_q = np.max(Q_table[next_state]) # FIX: Bellman uses max

        target = reward + discount_factor * best_next_q # FIX: gamma, not alpha

        if done:
            target = reward

        Q_table[state, action] = ( # FIX: update the current state's Q-value
            Q_table[state, action]
            + learning_rate *
            (target - Q_table[state, action])
        )

        state = next_state

    if done:

```

```

        break

print("Learned Q-table:")
print(np.round(Q_table, 3))

action_symbols = {
    0: "←",
    1: "↓",
    2: "→",
    3: "↑"
}

grid = env.unwrapped.desc

print("\nLearned Delivery Robot Policy")
print("-----")

for row in range(4):

    line = ""

    for col in range(4):

        state = row * 4 + col
        cell = grid[row][col].decode()

        if cell == "S":
            symbol = "S"
        elif cell == "G":
            symbol = "G"
        elif cell == "H":
            symbol = "X"
        else:
            best_action = np.argmax(Q_table[state]) # FIX: argmax
            symbol = action_symbols[best_action]

        line += f"[ {symbol} ] "

    print(line)

state, info = env.reset()

start_state = state
path = [state]
total_reward = 0

for step in range(30):

```

```

    action = np.argmax(Q_table[state]) # FIX: greedy = argmax

    next_state, reward, terminated, truncated, info = env.step(action)

    total_reward += reward
    path.append(next_state)

    state = next_state

    if terminated or truncated: # FIX: stop on either
        break

print("\nStart state:", start_state)
print("Path followed:", path)
print("Total reward:", total_reward)

goal_state = 12 # FIX: goal G is state 12 (row 3, col 0)

if state == goal_state:
    print("Delivery successful!")
else:
    print("Delivery unsuccessful.")

env.close()

```

Learned Q-table:

```

[[0.857 0.902 0.815 0.857]
 [0.857 0.857 0.774 0.815]
 [0.815 0.    0.735 0.774]
 [0.774 0.698 0.735 0.735]
 [0.902 0.95  0.857 0.857]
 [0.902 0.901 0.    0.814]
 [0.    0.    0.    0.   ]
 [0.    0.417 0.697 0.735]
 [0.95  1.    0.902 0.902]
 [0.95  0.    0.    0.857]
 [0.895 0.    0.    0.   ]
 [0.768 0.    0.    0.   ]
 [0.    0.    0.    0.   ]
 [0.8   0.    0.    0.   ]
 [0.    0.    0.    0.   ]
 [0.    0.    0.    0.   ]]

```

Learned Delivery Robot Policy

```

-----
[ ↓ ] [ ← ] [ ← ] [ S ]
[ ↓ ] [ ← ] [ X ] [ ↑ ]

```

```
[ ↓ ] [ ← ] [ ← ] [ ← ]  
[ G ] [ ← ] [ ← ] [ ← ]
```

Start state: 3

Path followed: [3, 2, 1, 0, 4, 8, 12]

Total reward: 1

Delivery successful!